

Pulse EDU

Final Development Clarification — Answers

Responses prepared for launch planning. Production site: <https://pulseedu.pulsekinetics.us/>

This document answers the pre-launch questionnaire. I focus on what the platform provides today, what you must configure on your infrastructure, and items that need a conscious decision before go-live.

Must know before launch

1) Text alerts for Request Pullout are sent by email (Resend), not SMS — SMS would require a separate provider such as Twilio or Amazon SNS and is not built yet. 2) ClassLink / OneRoster automatic roster sync is scaffolded but not live; schools onboard with the in-app checklist and CSV Data Importer until sync is completed. 3) You should own production credentials (database, domain, email API keys).

1. Documentation

At launch I will deliver a PDF package covering security overview, database and tenancy model, hosting configuration, email notifications, backup expectations, school onboarding, troubleshooting, system administration, and an FAQ for schools and districts.

Formats: PDF as the primary deliverable; Word copies on request.

Already in the product today:

- Settings !' Onboarding Checklist with downloadable PDF.
- Technical runbook in the repository (replit.md): stack, env vars, multi-tenancy rules.
- Optional generated staff guides (Core Team / Teacher) from the scripts package.

Formal legal FERPA/COPPA opinions are the district's responsibility with counsel; this documentation describes technical and operational controls only.

2. Security, Privacy, and Compliance

Data location and protection

- Student and staff records live in PostgreSQL (your DATABASE_URL). Media uploads use object storage paths you configure (PRIVATE_OBJECT_DIR / PUBLIC_OBJECT_SEARCH_PATHS).
- Every school-scoped query is filtered by school_id. SuperUsers operate within their district, not across unrelated districts.
- Staff sign in with server-side sessions stored in PostgreSQL; parents use a separate parent-portal login (invite-based, bcrypt passwords, rate limiting).
- The API uses Helmet (CSP in production), CORS allowlists, CSRF on mutating requests, and structured HTTP logging (Pino).
- For case AI consistency checks, student names are replaced with aliases before any model sees the bundle.

Encryption

- In transit: HTTPS on the public site (TLS at your reverse proxy / host).
- At rest: Provided by your database and storage host (enable encryption on PostgreSQL and object storage). The app does not add separate field-level encryption for roster columns.

Access control

- Role flags on staff (Admin, Dean, MTSS Coordinator, ISS, PBIS, etc.) gate routes and UI.
- Parents only see students linked to their account; staff only see data for the active school context.

Backups and audit

- Database backup and point-in-time recovery follow your PostgreSQL provider's policy. I recommend a documented restore drill before launch.
- The application maintains audit trails for key domains (e.g., investigations/cases, safety plans). HTTP logs support troubleshooting; a full SOC/SIEM is not included.

FERPA (operational alignment)

- Pulse EDU stores education records used by school officials: roster identifiers, behavior, MTSS/ISS, accommodations, and parent-visible HeartBEAT summaries where enabled.
- Access is limited to authenticated staff in the correct school and to parents for linked students only.

COPPA (operational alignment)

- The parent portal is for adults (invite-based accounts), not student self-registration.
- Day-to-day student interaction is staff-operated (classroom tools, kiosk/signage under school control).
- District policy and counsel determine any parental consent requirements; I document what the parent portal displays and how invites work.

Questions stakeholders often ask

- Where is data hosted and who can access it?
- Can one school see another school's data? (No, when tenancy is configured correctly.)
- How do staff get access when someone leaves? (Deactivate staff; manage the sign-in allowlist.)
- How are parents invited and what can they see?
- Is SMS used for alerts? (No — see Messaging section.)
- Is AI used on student information? (Case consistency check uses aliased bundles only.)

3. ClassLink / OneRoster Integration

Current status

Adapter structure exists in the codebase (OneRoster rostering and optional ClassLink SSO), with per-school settings in `district_integrations`. Automatic sync of students, staff, and room assignments is not production-ready yet. Until sync is finished, use CSV Data Importer and the in-app onboarding checklist.

When live sync is delivered, it will:

- Connect via OneRoster API credentials stored as environment variables (not in the database row).
- Sync students, staff, and room assignments on a scheduled basis.
- Require district approval of the ClassLink application and redirect URIs for SSO if used.

District setup: approved ClassLink app, credentials in your server environment, and integration row per school. I will document sync status fields (last sync time/status) in the admin guide.

4. Notifications and Messaging

SMS is not live

Request Pullout alerts go to designated staff by email today, not text message. The codebase reserves hooks for a future SMS provider (comments reference Twilio). If you require text messaging at launch, that is additional work: choose Twilio or Amazon SNS, provision numbers, comply with carrier rules, and budget per-message cost separately from email.

What works today (email via Resend)

- When a teacher submits a Request Pullout, active staff in that school with Admin, Dean, MTSS Coordinator, or ISS Teacher role and a valid email receive a dispatch email.
- Other email flows include parent invites, PBIS milestone emails to parents, pullout arrival/return messages, and optional intervention reminder digests when `EMAIL_REMINDERS_ENABLED=true`.
- Configure `RESEND_API_KEY` and `RESEND_FROM_EMAIL` on the production server.

Costs

- Email: per your Resend plan and send volume.
- SMS: not applicable until a provider is integrated; estimate after provider and volume are known.

Recipient control

- Dispatch recipients are determined by staff role flags and active status in that school. There is no separate SMS recipient admin screen yet.

5. New School Onboarding

Onboarding is built into the app: Settings !' Onboarding Checklist, organized in five phases (Identity & Access, Schedule & Operations, Behavior & PBIS, Interventions & MTSS, Family & Outreach). Each step links to the correct settings screen and can auto-detect completion or be marked done manually. A printable PDF is available from the same screen.

Typical steps

- Create the school (SuperUser !' Tenancy) under your district.
- Configure branding, staff allowlist, staff directory/default rooms, and locations.
- Set a default bell schedule before turning on Hall Pass Queue.
- Configure PBIS, MTSS, parent portal sections, and send parent invites as needed.
- Import or enter roster data (CSV Data Importer until ClassLink sync is live).

Timeline and separation

- Typical setup: about one to two weeks for a motivated admin team; longer if waiting on external roster feeds.
- Data is isolated per `school_id`; permissions are managed per school via staff roles.

The onboarding checklist in the application is the authoritative checklist; no separate paper-only process is required.

6. Launch Support

Support terms (hours, response times, 30/60/90-day windows, and fees) are defined in our project agreement, not in the application itself.

- I recommend email for day-to-day issues and scheduled screen shares during the first weeks after launch.
- Critical production outages should have a named contact and escalation path in the support agreement.
- Included vs excluded scope (bug fixes on delivered features vs new modules) should match the signed statement of work.

7. Safe Changes vs Developer-Managed Areas

School admins can safely adjust settings content: branding, locations, PBIS reasons, bell schedules, allowlists, and onboarding steps that only touch school data.

Changes that require a developer:

- Database schema (lib/db/src/schema), API routes, authentication, CSRF, and session configuration.
- New features, integrations (ClassLink live sync, SMS), and object-storage ACL logic.

Any new screen or table must include `school_id` on tenant data and use existing auth helpers so data cannot leak across schools.

8. Ownership, Access, and Sustainability

Pulse EDU is deployed on infrastructure you own and control. I build, secure, and configure the application; you retain the production accounts.

- You should own: domain/DNS, application server, PostgreSQL database, Resend account (email), and repository access.
- At handoff I provide: source code, environment variable list (`.env.example`), deployment notes, and this clarification package.
- Credentials to transfer: `DATABASE_URL`, `SESSION_SECRET`, `RESEND_API_KEY`, storage paths, and any future integration keys (ClassLink, Twilio/SNS).
- Avoid leaving production-only secrets on a developer personal account.

Another developer can maintain the system with Node.js 24, pnpm, Express API, React client, and Drizzle ORM; start with multi-tenancy (`scope.ts`) and authentication.

- Third-party services in use or planned: PostgreSQL hosting, Resend (email), optional Twilio or AWS SNS if SMS is added, ClassLink when sync is completed.

9. Launch Readiness Checklist

Use this table to confirm status before go-live:

Item	Status / action
Security controls (app layer)	Ready — review your host encryption and HTTPS
FERPA / COPPA legal sign-off	District counsel
ClassLink / OneRoster auto-sync	Not production-ready — use CSV + onboarding
Hosting (PostgreSQL + Node app)	Confirm your server and DATABASE_URL
Email notifications (Resend)	Configure keys; test pullout dispatch email
SMS notifications	Not available — email only unless SMS is scoped
User role testing	Required — test each role against your matrix
Request Pullout !' staff alert	Test email to Admin/Dean/MTSS/ISS
School onboarding	Use in-app checklist + PDF
Documentation package	This PDF + onboarding export
Support plan	Per project agreement
Database backups	Enable on provider; run restore drill
Known limitations	No SMS; no live ClassLink sync; email reminders off unless enabled
Final walkthrough	Schedule after items above pass

Request Pullout workflow (confirmed behavior)

Teacher submits Request Pullout !' system sends email to active dispatch-role staff in that school (Admin, Dean, MTSS Coordinator, ISS Teacher) who have valid email addresses !' send is tracked on the pullout record (idempotent). Text messaging is not part of this workflow until an SMS provider is integrated.

